

YOUR AWS APP IS SLOW!

WHERE DO YOU LOOK FIRST?



METRICS • LOGS • ALARMS

WHAT IS AWS CLOUDWATCH?



Amazon CloudWatch is AWS's monitoring and observability service. It collects **metrics**, logs, and **events** from your resources and applications to help you **monitor**, **detect**, and **react**.



KEY CONCEPTS

METRICS



Numerical data collected over time. Helps you understand system performance.

LOGS



Detailed records of events generated by applications and AWS services.

ALARMS



Set thresholds on metrics and get notified or take automatic action.

DASHBOARDS



Visualize metrics and logs in a centralized view.



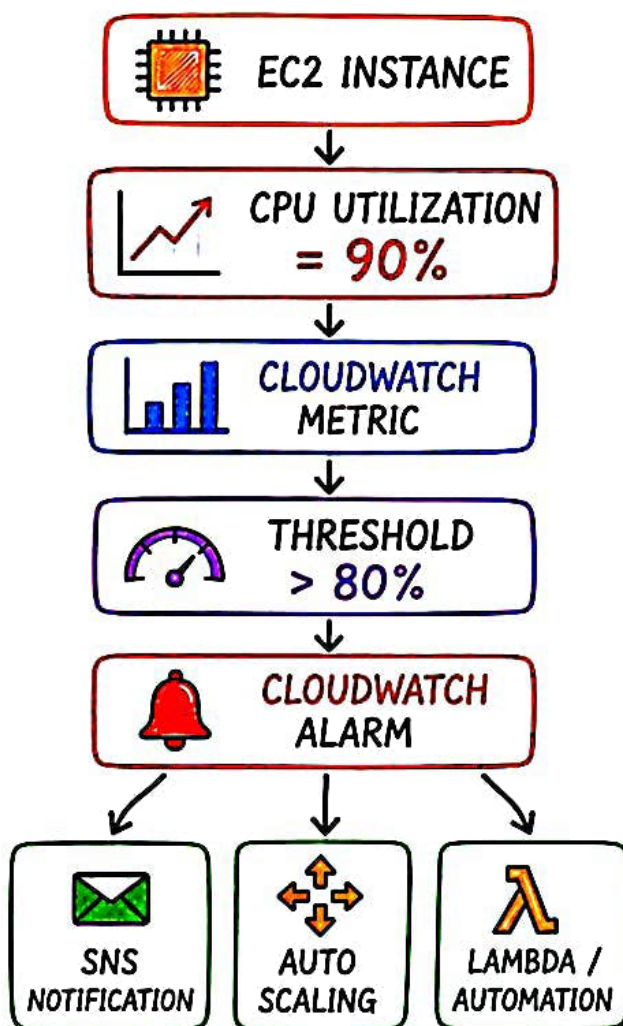
CLOUDWATCH = **OBSERVE** → **DETECT** → **ALARM** → **ACT**

Your eyes inside the cloud!



CPU HITS 90%.

NOW WHAT?



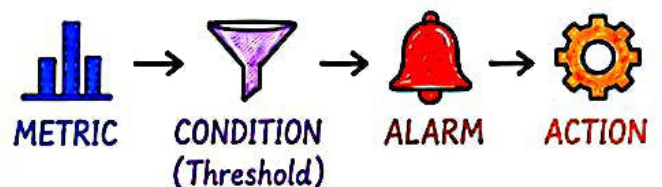
ALARM STATES

	OK	Everything is working fine.
	ALARM	The metric is breaching the threshold.
	INSUFFICIENT DATA	Not enough data to determine the state.

EXAMPLE ALARM

If CPUUtilization > 80% for 5 minutes, then trigger an **ALARM**.

HOW ALARMS WORK



METRIC → CONDITION → ALARM → ACTION

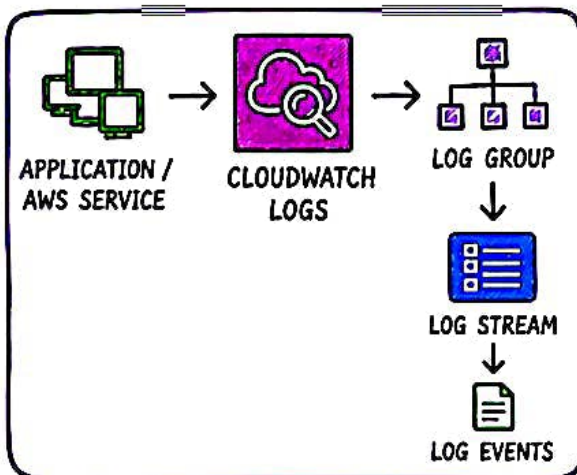
Monitor smartly. Detect early. Act fast!



APPLICATION FAILED? FIND THE ERROR!

CLOUDWATCH LOGS

Centralized logging for your applications and AWS services.



SAMPLE LOG EVENTS

10:31:12	INFO	Request received
10:31:13	INFO	Validating input
10:31:14	INFO	Connecting to DB
10:31:16	ERROR	DB connection timeout
10:31:16	ERROR	Exception occurred
10:31:17	ERROR	HTTP 500 Internal Server Error
10:31:18	WARN	Retrying request
10:31:20	INFO	Request failed

LOGS INSIGHTS

Query and analyze logs in seconds!

- ☒ Where are the errors?
- ☒ When did they start?
- ☒ Which requests are failing?



WHY LOGS MATTER?

- See what actually happened.
- Troubleshoot issues faster.
- Audit and compliance.
- Create metrics from logs.
- Set alarms on critical events.

METRICS TELL YOU WHAT?



Something is wrong
(high CPU, high errors, etc.)

LOGS TELL YOU WHY?



What exactly happened
and where it failed.

COMMON CLOUDWATCH LOGS SOURCES



LAMBDA
FUNCTION



EC2
INSTANCES



ALB / NLB



RDS
DATABASE



S3
ACCESS LOGS



API
GATEWAY



ECS / EKS
CONTAINERS



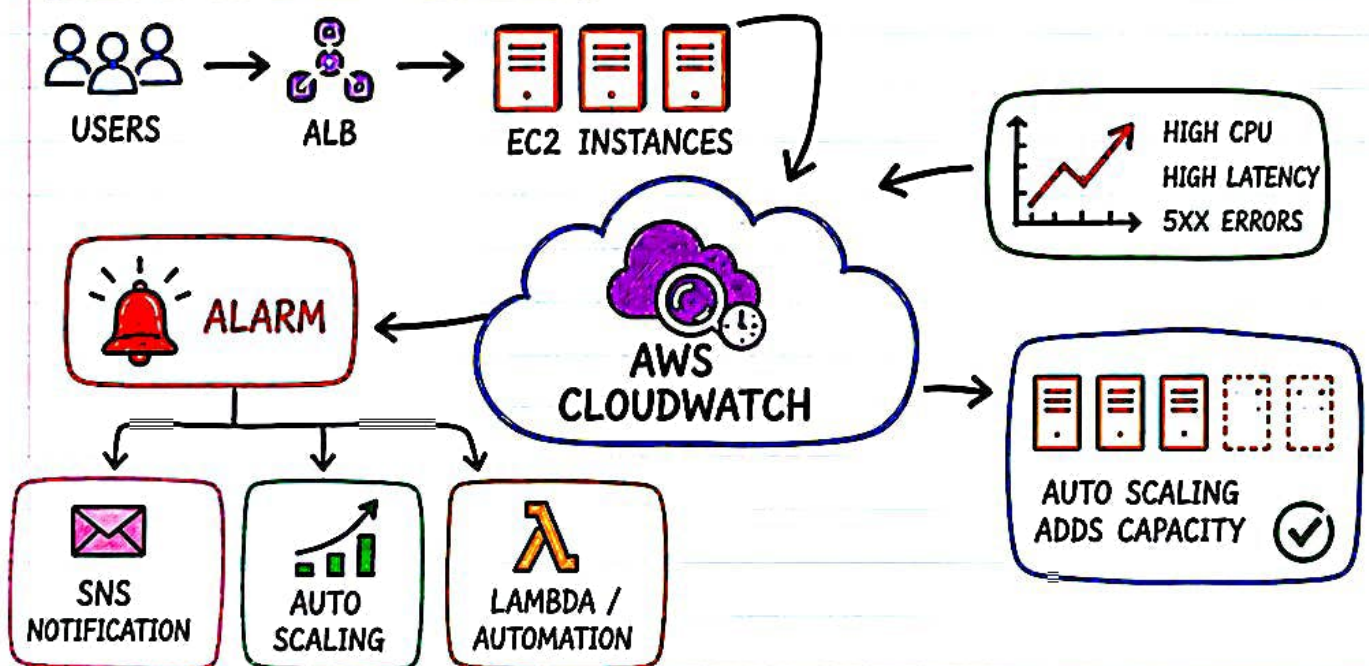
GOOD LOGS = QUICKER ROOT CAUSE ANALYSIS
BETTER LOGS = STRONGER SYSTEMS



DON'T JUST MONITOR. = REACT! =

CloudWatch helps you **OBSERVE** → **DETECT** → **ACT**

SCENARIO: TRAFFIC SPIKE!



= OTHER COMMON ACTIONS =



SNS

Alert your team



S3

Store logs



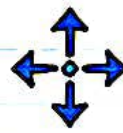
LAMBDA

Trigger automation



EVENTBRIDGE

Trigger workflows



AUTO SCALING

Scale resources

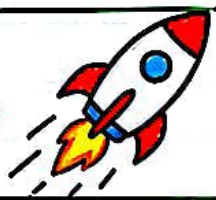


CHAT OPS

Slack / Teams alerts



GOOD MONITORING FINDS PROBLEMS.
GREAT MONITORING **FIXES PROBLEMS FAST!**



CPU IS NORMAL... BUT USERS SAY THE APP IS SLOW!

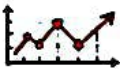


WHAT DO YOU CHECK?

1 CHECK CLOUDWATCH METRICS
 CPU, Memory, Disk, Network, Latency, Error Rates, etc.

2 CHECK ALB / APP METRICS
 TargetResponseTime, RequestCount, 4XX, 5XX

3 ANALYZE CLOUDWATCH LOGS
 Search errors, timeouts, exceptions in logs.

4 CORRELATE THE TIMELINE
 Match spikes in metrics with log errors.

5 CHECK ALARMS
 Are any alarms triggered? What do they indicate?

6 IDENTIFY THE BOTTLENECK
 Compute? DB? Network? External dependency?

7 FIX & IMPROVE MONITORING
 Resolve the issue and add better metrics & alerts.



INTERVIEW QUESTION

Q: CPU is normal, but users report high application latency. What would you check?



STRONG ANSWER

- ✓ Don't stop at CPU.
- ✓ Check latency metrics (ALB, TargetResponseTime, API GW, etc).
- ✓ Check 4XX / 5XX errors.
- ✓ Review application logs.
- ✓ Check downstream services (DB, Cache, External APIs).
- ✓ Analyze connections, timeouts and slow dependencies.
- ✓ Correlate the timeline and identify the real bottleneck.



METRICS



WHAT?

Tells you what is happening.

LOGS



WHY / DETAILS?

Shows you what actually happened.

ALARMS



WHEN TO REACT?

Notifies you when things go wrong.

DASHBOARDS



BIG PICTURE!

Visualize and monitor everything together.



SAVE THIS FOR YOUR AWS INTERVIEW!



YOUR WEBSITE IS UNDER ATTACK!



SQL Injection



XSS attacks



Bad Bots



Thousands of
requests hitting
/login

Big question:

HOW DO YOU STOP THEM
BEFORE THEY REACH YOUR
APPLICATION?

INTRODUCING

AWS WAF

Web Application Firewall



Users



Internet



AWS WAF



Your
Application

INSPECT → FILTER → PROTECT

⌈ AWS WAF EXPLAINED SIMPLY ⌋

AWS WAF (Web Application Firewall) protects your web applications and APIs by *inspecting* HTTP/HTTPS requests and *filtering unwanted or malicious traffic*.

HOW AWS WAF WORKS?



WHAT CAN AWS WAF INSPECT?

- IP Address
- URI Path
- HTTP Headers
- Query Strings
- Cookies
- Request Body

WAF TAKES ACTION

- ✓ ALLOW
- ✗ BLOCK
- 👁️ COUNT
- 🔒 CAPTCHA
- 🛡️ CHALLENGE

KEY BENEFITS

- ✓ Blocks common web exploits like SQLi, XSS
- ✓ Controls bots and abusive traffic
- ✓ Rate limiting & request filtering
- ✓ Protects APIs and web applications
- ✓ Works at **Layer 7** (application layer)



REMEMBER:

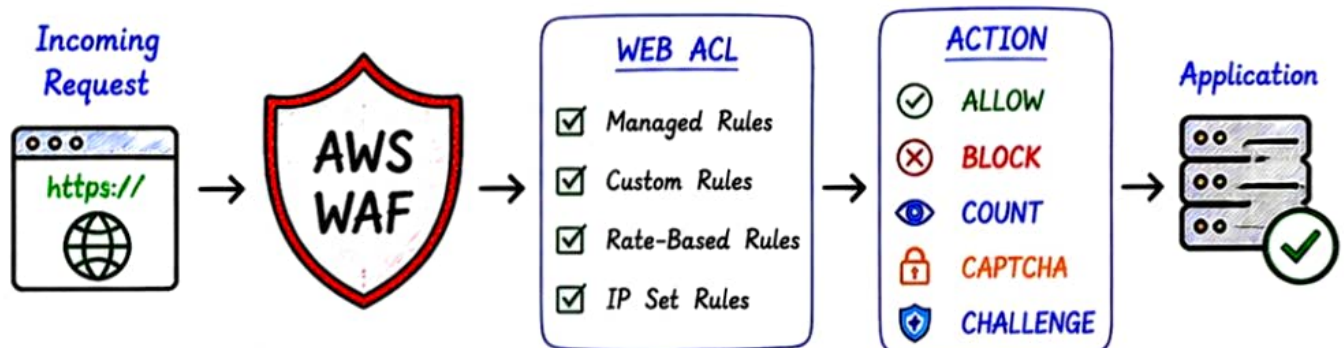
AWS WAF sits *in front* of your application and allows good traffic while *blocking* the bad!



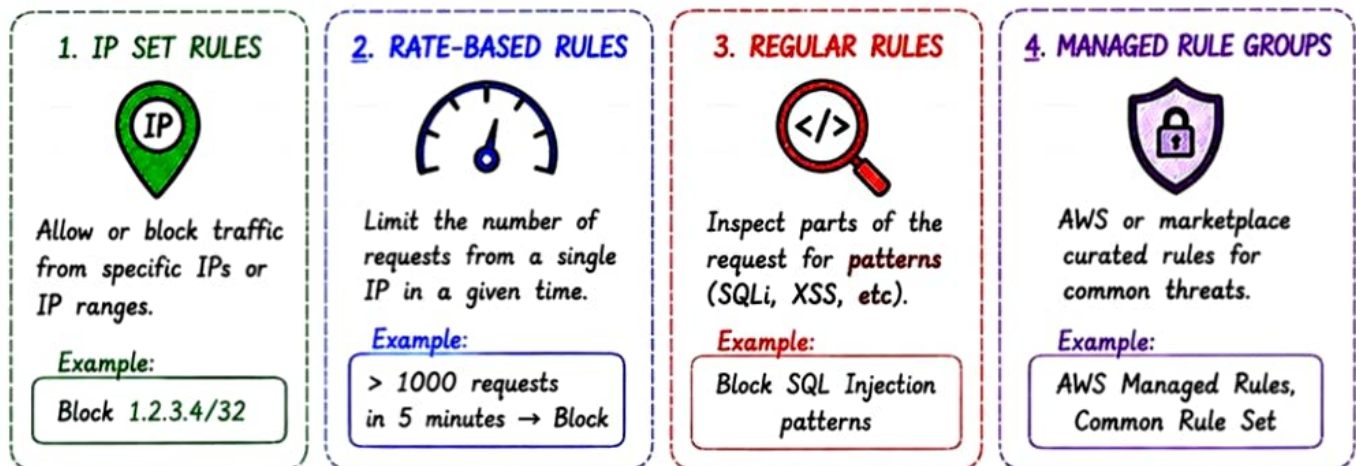
AWS WAF RULES & WEB ACL

AWS WAF uses a Web ACL (Web Access Control List) which contains rules to inspect requests and take action.

HOW AWS WAF EVALUATES A REQUEST



TYPES OF RULES YOU CAN CREATE



RULE PRIORITY (TOP TO BOTTOM)

- 1 WAF evaluates rules in **priority order**.
- 2 First matching rule = **action** is applied.
- 3 If no rule matches → **default action**.

DEFAULT ACTION

If no rule matches:

- ✓ ALLOW by default
- ✗ You can set **DEFAULT BLOCK**



💡 WEB ACL + SMART RULES = STRONG PROTECTION FOR YOUR APPLICATIONS

REAL PRODUCTION SCENARIO

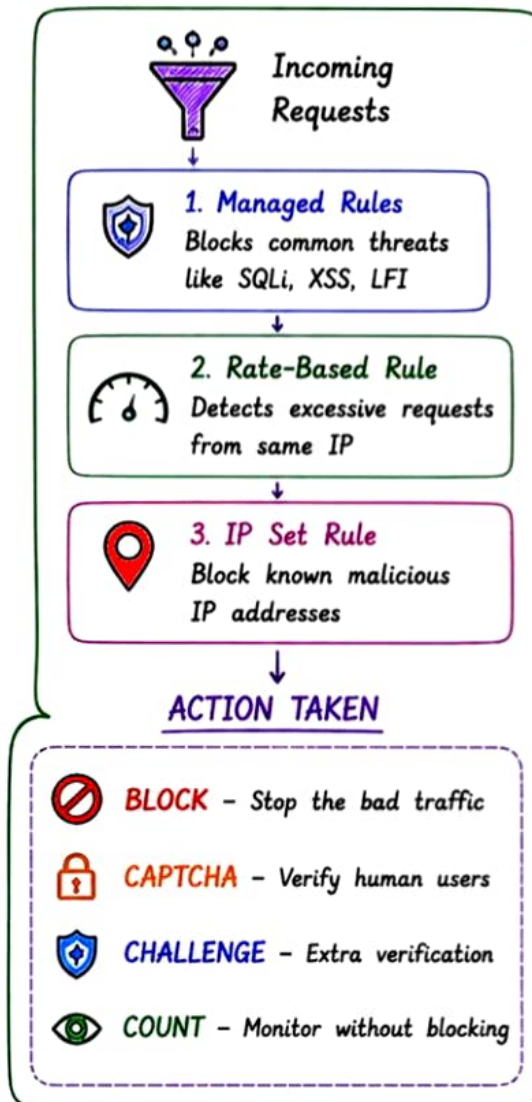
YOUR /LOGIN ENDPOINT IS GETTING HAMMERED!

Attackers are sending thousands of requests to /login to guess passwords (Credential Stuffing Attack)

THE ATTACK FLOW



HOW AWS WAF PROTECTS YOU



WHY RATE-BASED RULES?

- Limit abusive requests from single IP
- Protect sensitive endpoints like /login, /signup, /api
- Stop bots & credential stuffing attempts
- Reduce load on your application

BEST PRACTICE

- ✓ Use AWS Managed Rules
- ✓ Add Rate-Based Rules
- ✓ Monitor in COUNT mode before blocking
- ✓ Tune rules to reduce false positives
- ✓ Enable logging & alerts



REMEMBER:

Don't wait until malicious traffic reaches your servers.
Stop it at the edge with **AWS WAF!**



AWS WAF = BLOCK BAD TRAFFIC EARLY, KEEP YOUR APP SAFE!

WAF vs SHIELD vs SECURITY GROUPS

These 3 services protect your applications, but they work at **DIFFERENT LAYERS!**

	AWS WAF 	AWS SHIELD 	SECURITY GROUPS 
 What it is	Web Application Firewall	Managed DDoS Protection	Virtual Firewall for Your Resources
 Protects Against	Malicious web requests (SQLi, XSS, Bots, etc.)	DDoS attacks (Volumetric attacks)	Unauthorized network access
 Layer	Layer 7 (Application Layer)	Layer 3, 4 & 7 (Network + Transport)	Layer 3 & 4 (Network)
 Operates on	HTTP/HTTPS requests	Network traffic	IP addresses, Ports, Protocols
 Example	Block SQL Injection in /login	Stop a DDoS attack flood	Allow TCP 443 from trusted IP
 Key Point	Inspects & filters web requests	Absorbs & mitigates DDoS traffic	Controls traffic to your resources

EASY ANALOGY

AWS SHIELD



Protects the building from big attacks (DDoS)

AWS WAF



Security guard checks every visitor (web request) before entry

SECURITY GROUPS



Controls which doors (ports) are open to trusted people only

HOW THEY WORK TOGETHER



SUMMARY: Use **SHIELD** to stop attacks, **WAF** to filter bad requests, and **SECURITY GROUPS** to control network access.

TOGETHER = STRONG SECURITY!

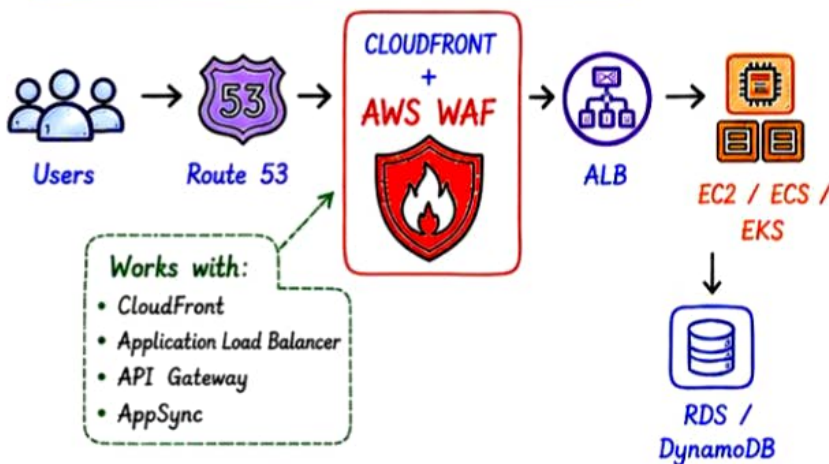


AWS WAF INTERVIEW CHEAT SHEET



Place AWS WAF in front of your application and **stop threats** before they reach your servers!

TYPICAL PRODUCTION ARCHITECTURE



INTERVIEW QUESTION

Q: Your production website is receiving thousands of malicious requests against **/login**. What would you do?

STRONG ANSWER APPROACH

1. Deploy / verify AWS WAF in front of the application (CloudFront or ALB).
2. Enable AWS Managed Rules to block common threats like **SQLi**, **XSS**, **LFI**, etc.
3. Add **Rate-Based Rules** to limit abusive requests on **/login** and other sensitive endpoints.
4. Use **COUNT** mode first to understand traffic patterns and avoid false positives.
5. Monitor WAF metrics, logs & tune rules.
6. Consider **CAPTCHA** / **Challenge** for suspicious traffic.
7. Combine with **AWS Shield** for DDoS protection.

BEST PRACTICES

- ✓ Start with AWS Managed Rules
- ✓ Use Rate-Based Rules for sensitive endpoints
- ✓ Protect important paths like **/login**, **/signup**, **/api**
- ✓ Use IP sets to allow/block specific ranges
- ✓ Use **CAPTCHA** / **Challenge** to stop bots
- ✓ Monitor & analyze in **COUNT** mode first
- ✓ Enable logging & alerts
- ✓ Regularly review & tune rules
- ✓ Combine with Shield for DDoS protection

WAF ACTIONS

- ✓ **ALLOW** - Let the request through
- ✗ **BLOCK** - Stop the request
- 👁️ **COUNT** - Log & monitor, don't block
- 🔒 **CAPTCHA** - Verify human users
- 🛡️ **CHALLENGE** - Extra verification before allowing

KEY TAKEAWAY

AWS WAF =
INSPECT → **FILTER** → **PROTECT**
 The earlier you block, the safer your applications!



SECURE AT THE EDGE. PROTECT WHAT MATTERS.

